



RAPPORT DE PROJET DE DATA ENGINEERING

Projet 13 : Tableau de bord de qualité de données avec profilage et traçabilité

Réalisé par :

Aboubakr TAHIR

Younes LYAZIDI

Encadré par :

Pr. Hamid HRIMECH

1^{er} juin 2026

Table des matières

1	Introduction et Contexte	2
1.1	Contexte Général	2
1.2	La Problématique des "Silent Failures"	2
1.3	Objectifs du Projet	2
1.4	Notre Approche : Le DataOps de Bout-en-Bout	3
2	Architecture Globale de la Plateforme	4
3	Choix Technologiques	6
4	Implémentation et Réalisations	8
4.1	Génération de Données et Chaos Engineering	8
4.2	Modélisation de la Base Source (OLTP)	8
4.3	Orchestration du Pipeline (Airflow DAG)	9
4.4	Profilage et Qualité des Données (Great Expectations)	10
4.5	Transformation et Architecture Medaillon (dbt)	10
4.6	Machine Learning et MLOps (Détection d'anomalies)	10
4.7	Création de la Couche Gold (Quarantaine et Faits)	11
4.8	Traçabilité et Lignage (Marquez / OpenLineage)	11
5	Restitution Visuelle : Tableaux de Bord	13
5.1	Le Tableau de Bord de Performance Financière	13
5.2	Le Tableau de Bord des Alertes Fraudes	13
6	Conclusion	15

1 Introduction et Contexte

1.1 Contexte Général

Dans les environnements d'entreprise modernes, la donnée est considérée comme le nouvel or noir. Elle est le moteur central des prises de décision stratégiques, de l'intelligence artificielle et des opérations courantes. Cependant, avec l'augmentation exponentielle du volume, de la variété et de la vélocité des données (les "3V" du Big Data), les infrastructures de données (Data Warehouses, Data Lakes) deviennent extrêmement complexes. Une DSI (Direction des Systèmes d'Information) fait face à un défi majeur : garantir que la donnée stockée est saine, fraîche, exacte et exploitable.

1.2 La Problématique des "Silent Failures"

L'un des défis majeurs dans l'ingénierie des données est le phénomène des pannes silencieuses (*Silent Failures*). Contrairement à une erreur de syntaxe ou un serveur hors ligne qui génère immédiatement une alerte rouge, une panne silencieuse est invisible.

Par exemple, si une application source subit une mise à jour mineure qui change le format d'une colonne (ex : "Montant" passe de *float* à *string*, ou génère des montants négatifs), le pipeline d'extraction (ETL) peut continuer à fonctionner sans générer d'erreur technique. La donnée corrompue traverse alors les différentes couches de l'entrepôt.

Conséquences directes :

- Les tableaux de bord décisionnels (BI) affichent des métriques erronées.
- Les algorithmes de Machine Learning s'entraînent ou prédisent sur des données fausses, entraînant un phénomène de *Data Drift* ou de *Concept Drift*.
- La DSI perd des semaines à tracer manuellement la source de l'erreur dans un écosystème complexe.

1.3 Objectifs du Projet

L'objectif de ce projet est de répondre à cette problématique en développant une **Plateforme d'Observabilité des Données** (*Data Observability Platform*).

Le cahier des charges initial (Projet 13) exigeait :

1. Le développement d'un framework de tests de qualité (nulls, unicité, fraîcheur) via **Great Expectations**.
2. La traçabilité et la lignée de la donnée via **Marquez / OpenLineage**.
3. L'orchestration avec **Apache Airflow** et les transformations avec **dbt**.
4. La visualisation temps réel dans un dashboard avec **Apache Superset**.
5. Un aspect avancé de détection d'anomalies sur les métriques de qualité.

1.4 Notre Approche : Le DataOps de Bout-en-Bout

Pour résoudre cette problématique, nous avons adopté une approche **DataOps** globale. Au lieu de nous limiter à de simples requêtes SQL de vérification, nous avons conçu une architecture robuste qui intègre la qualité comme "citoyen de première classe" à chaque étape du flux.

Afin de prouver l'efficacité de notre solution, nous avons implémenté du **Chaos Engineering** : notre pipeline simule des transactions bancaires et *injecte intentionnellement* des corruptions structurelles et des fraudes comportementales complexes. Nous démontrons ainsi comment nos barrières de qualité (Great Expectations) bloquent les erreurs basiques, tandis que nos algorithmes d'Intelligence Artificielle (Isolation Forest et Autoencoders) s'occupent d'identifier les anomalies indétectables par l'humain.

2 Architecture Globale de la Plateforme

Afin de traiter les données de manière optimale, nous avons conçu une architecture hybride, combinant un environnement local conteneurisé (Docker) pour l'orchestration, et un Data Warehouse Cloud (Google BigQuery) pour la puissance de calcul.

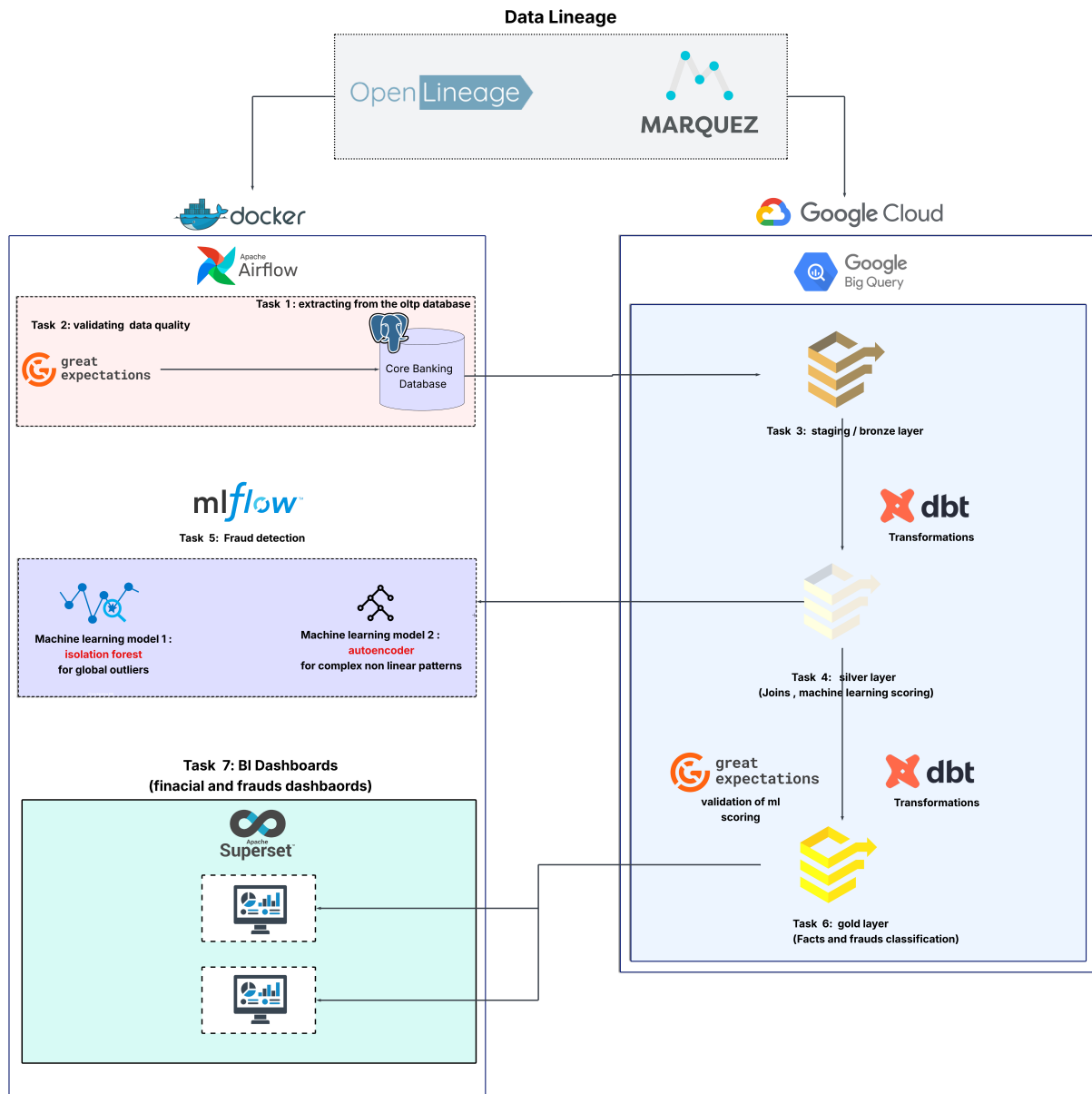


FIGURE 1 – Architecture haut-niveau de la plateforme (Générée avec Lucidchart)

L'architecture se décompose en trois piliers fondamentaux :

1. **La source de données transactionnelle (OLTP) :** Représentée par une base de données PostgreSQL, simulant le cur du système d'information de la banque.
2. **Le Data Warehouse Analytique (OLAP) :** Google BigQuery, structuré selon le standard industriel de l'Architecture Medaillon (Bronze, Silver, Gold).

3. **L'environnement DataOps (Conteneurs Docker)** : Hébergeant Airflow (l'orchestrateur central), Great Expectations (la qualité), dbt (la transformation), MLflow (le Machine Learning), Marquez (la traçabilité) et Superset (la visualisation).

Le flux de la donnée est entièrement automatisé. Airflow déclenche l'extraction depuis Postgres, valide la qualité, pousse vers BigQuery, déclenche dbt pour construire la couche analytique, puis invoque nos modèles de Machine Learning pour scorer la donnée, et enfin expose les résultats sur Superset.

3 Choix Technologiques

Chaque outil a été sélectionné pour répondre à un standard de l'industrie de la donnée (Modern Data Stack) :

- **Docker** : Socle technologique permettant de conteneuriser toute l'infrastructure, garantissant une portabilité totale de la solution.



- **Apache Airflow** : C'est le chef d'orchestre. Il permet de définir des tâches sous forme de graphes orientés acycliques (DAG) via du code Python. Il gère les dépendances, les relances en cas d'échec (retries) et la planification.



- **Google BigQuery** : Data Warehouse serverless puissant, permettant de stocker d'immenses volumes de données et d'exécuter des transformations SQL à la vitesse de l'éclair, sans gérer d'infrastructure.



- **dbt (Data Build Tool)** : Standard du marché pour la transformation de données (le 'T' de l'ELT). Il permet de gérer les requêtes SQL comme du code logiciel (versioning, tests d'intégrité, modularité, documentation automatique).



- **Great Expectations (GX)** : Framework Python dédié à la validation des données. Il permet de traduire les règles métier ("le montant doit être positif", "l'ID doit être unique") en tests automatisés exécutés dans le pipeline.
- **Marquez & OpenLineage** : Serveur de métadonnées de référence pour tracer le lignage des données. Il collecte les métadonnées émises par Airflow et dbt pour dessiner un graphe d'impact de bout-en-bout.



- **MLflow & IA** : Outils de Machine Learning. Scikit-Learn et TensorFlow nous permettent de coder les modèles d'anomalie, tandis que MLflow assure le suivi des expériences, le versionnage des modèles et la sauvegarde des artefacts (MLOps).



- **Apache Superset** : Plateforme de Business Intelligence open-source, rapide, intuitive, et hautement intégrable avec les bases de données Cloud modernes pour la restitution visuelle.

4 Implémentation et Réalisations

4.1 Génération de Données et Chaos Engineering

Pour avoir un terrain de jeu réaliste, nous avons développé un script Python basé sur la librairie *Faker* qui peuple la base PostgreSQL.

L'approche novatrice réside dans le **Chaos Engineering**. Le script injecte intentionnellement des anomalies selon des règles strictes :

- **Les jours pairs du mois** : Injection d'erreurs structurelles (montants négatifs, canaux de transaction invalides, âges absurdes). Ces erreurs visent à tester la robustesse de notre couche de "Data Quality".
- **Les jours impairs du mois** : Injection de comportements frauduleux silencieux (micro-transactions répétitives très rapides, changements soudains de pays). Ces transactions sont structurellement parfaites (montants valides, IDs corrects), elles traversent donc les règles de qualité de base. Elles visent à tester nos algorithmes d'Intelligence Artificielle.

4.2 Modélisation de la Base Source (OLTP)

Avant d'être ingérées et transformées, les données de la banque sont modélisées dans notre base relationnelle PostgreSQL. Le schéma se compose de quatre tables principales hautement interconnectées :

- **transactions** : Table centrale contenant toutes les opérations financières (ID de transaction, montant, date, canal de paiement, localisation, adresse IP, tentatives de connexion, solde du compte).
- **accounts** : Informations sur les comptes des clients (ID du compte, nom du client, âge, profession, statut du compte).
- **devices** : Terminaux utilisés pour effectuer les transactions (ID du terminal, nom, type d'appareil, système d'exploitation).
- **merchants** : Les commerçants recevant les paiements (ID du commerçant, nom, catégorie d'activité, ville).

Ce schéma relationnel sert de fondation solide pour la génération et le croisement de nos données synthétiques bancaires réalistes.

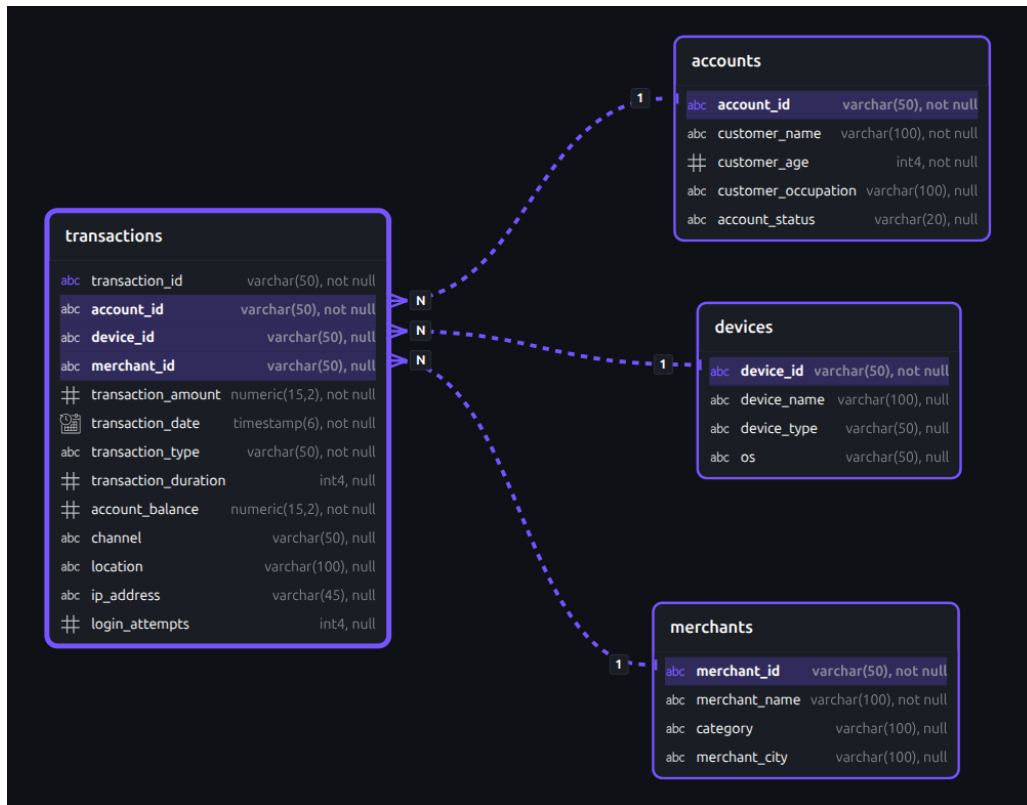


FIGURE 2 – Schéma relationnel (ERD) de la base de données PostgreSQL OLTP

4.3 Orchestration du Pipeline (Airflow DAG)

L'ensemble du pipeline est piloté par un DAG Airflow complexe composé de 9 étapes séquentielles. Ce DAG est planifié quotidiennement.

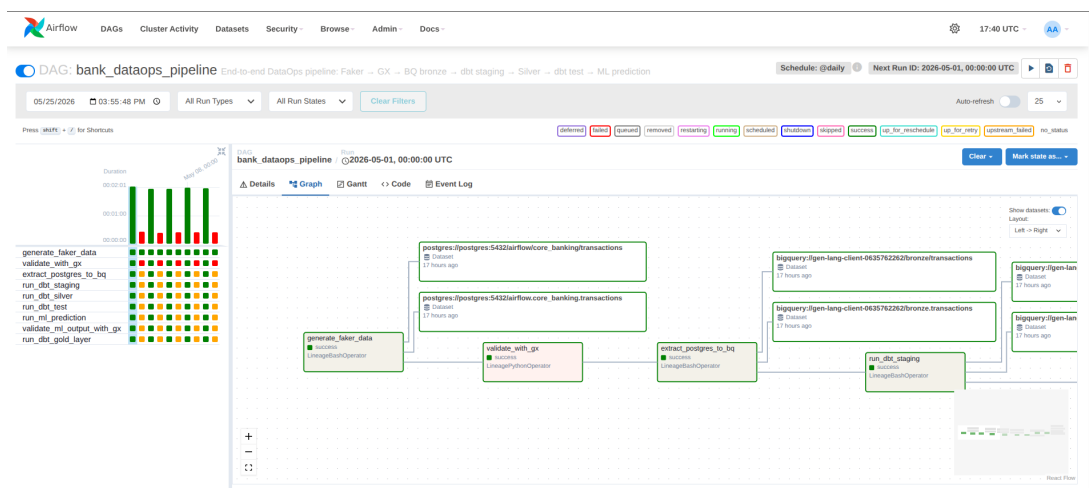


FIGURE 3 – Le graphe (DAG) exécuté dans Apache Airflow

Si une tâche échoue (par exemple, si les données sont corrompues), Airflow stoppe l'exécution de la suite du pipeline. Cela évite que des données fausses n'atteignent les tableaux de bord finaux, répondant directement à la problématique des pannes silencieuses.

4.4 Profilage et Qualité des Données (Great Expectations)

La qualité est vérifiée en amont (sur la source Postgres) et en aval (sur les prédictions ML). Nous avons défini des "Suites d'Expectations" (règles) via **Great Expectations**.

Exemples de contrôles appliqués :

- `expect_column_values_to_not_be_null` (Unicité des IDs).
- `expect_column_values_to_be_between` (Montant entre 0 et 100 000\$).
- `expect_column_values_to_be_in_set` (Canaux limités à ATM, ONLINE, MOBILE, IN_BRANCH).

Lors des jours "pairs", Great Expectations détecte nos anomalies, fait échouer la tâche Airflow de validation, et alerte immédiatement l'équipe d'ingénierie.

4.5 Transformation et Architecture Medaillon (dbt)

Une fois les données brutes ingérées dans Google BigQuery (couche **Bronze**), **dbt** prend le relais. dbt exécute des requêtes SQL pour nettoyer, standardiser et croiser les tables (Transactions, Clients, Appareils, Commerçants). Le résultat est stocké dans la couche **Silver**. dbt exécute également ses propres tests de schéma (intégrité référentielle, non-nullité) pour assurer que les jointures n'ont pas dupliqué de lignes.

Search tables or columns

silver_enriched_transactions

abc

transaction_id

STRING, null

abc

account_id

STRING, null

abc

device_id

STRING, null

abc

merchant_id

STRING, null

abc

ip_address

STRING, null

abc

transaction_amount

NUMERIC(10, 2), null

abc

transaction_date

TIMESTAMP, null

abc

transaction_type

STRING, null

abc

transaction_duration

INTEGER, null

abc

account_balance

NUMERIC(10, 2), null

abc

login_attempts

INTEGER, null

abc

channel

STRING, null

abc

location

STRING, null

abc

customer_age

INTEGER, null

abc

customer_occupation

STRING, null

abc

tx_fier

INTEGER, null

abc

tx_day_of_week

INTEGER, null

abc

account_tx_count_24h

INTEGER, null

abc

account_avg_amount_7d

NUMERIC(10, 2), null

abc

merchant_tx_count_7h

INTEGER, null

abc

amount_vs_avg_ratio

NUMERIC(10, 2), null

silver_scored_transactions

abc

transaction_id

STRING, null

abc

anomaly_score

FLOAT64, null

abc

is_anomaly

BOOLEAN, null

abc

threshold_used

BOOLEAN, null

abc

device_id

STRING, null

abc

merchant_id

STRING, null

abc

ip_address

STRING, null

abc

transaction_date

TIMESTAMP, null

FIGURE 4 – Couche Silver (Vue BigQuery)



All columns



Search tables or columns

gold_fact_transactions

abc	transaction_id	STRING, null
abc	account_id	STRING, null
abc	merchant_id	STRING, null
#	transaction_amount	NUMERIC(10), null
	transaction_date	TIMESTAMP, null
abc	transaction_type	STRING, null
abc	channel	STRING, null
abc	location	STRING, null

gold_quarantine_transactions

abc	transaction_id	STRING, null
abc	account_id	STRING, null
#	transaction_amount	NUMERIC(10), null
	transaction_date	TIMESTAMP, null
#	anomaly_score	FLOAT64, null
#	threshold_used	BOOLEAN, null
abc	ip_address	STRING, null
abc	device_id	STRING, null

FIGURE 5 – Couche Gold (Vue BigQuery)

4.6 Machine Learning et MLOps (Détection d'anomalies)

Pour l'aspect avancé de ce projet (la détection d'anomalies comportementales invisibles aux règles strictes), nous avons développé deux modèles d'IA complémentaires exécutés sur la couche Silver :

1. **Isolation Forest (Scikit-Learn)** : Modèle arborescent idéal pour isoler rapidement les *outliers* globaux dans la distribution des données (ex : un montant inhabituellement haut pour un jeune client).

2. **Autoencoder (TensorFlow / Keras)** : Réseau de neurones profond. Il apprend à compresser et reconstruire les transactions "normales". Si une transaction est difficile à reconstruire (erreur de reconstruction élevée), elle est classée comme anomalie. Idéal pour capter des corrélations non-linéaires complexes (ex : fraude coordonnée).

Ces modèles attribuent un "Score d'anomalie" à chaque transaction. Si le score dépasse un seuil dynamique, la transaction est taguée comme frauduleuse.

Toute cette intelligence artificielle est tracée avec **MLflow**, qui sauvegarde les hyperparamètres, la fonction de perte (loss) des réseaux de neurones, et le fichier sérialisé du modèle (Pickle) dans notre volume partagé, assurant une parfaite mise en production (MLOps).

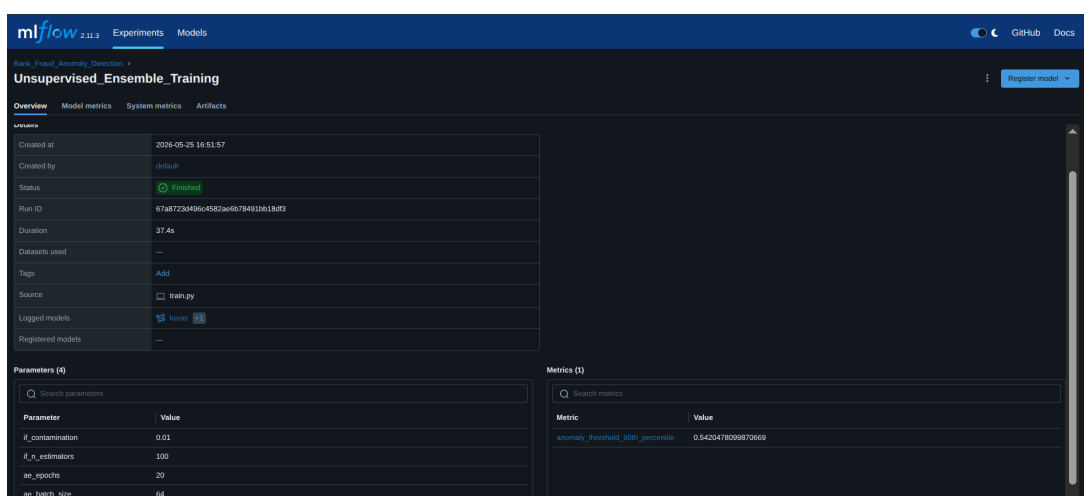


FIGURE 6 – Interface MLflow (Tracking des expérimentations MLOps)

4.7 Création de la Couche Gold (Quarantaine et Faits)

Après le passage des modèles ML, dbt intervient de nouveau pour scinder les données en deux tables dans la couche **Gold** :

- **Gold Fact Transactions** : Les transactions saines et validées, prêtes à être analysées par la finance.
- **Gold Quarantine** : Les transactions identifiées comme frauduleuses par l'IA, nécessitant une révision humaine. C'est l'équivalent de notre mécanisme "d'auto-réparation" (ségrégation des données malades).

4.8 Traçabilité et Lignage (Marquez / OpenLineage)

Répondant à l'objectif de traçabilité, la plateforme intègre **OpenLineage**. Chaque étape d'Airflow et chaque modèle dbt envoie un message télémétrique en temps réel (un événement JSON) au serveur **Marquez**. Marquez agrège ces informations pour construire

un graphe visuel interactif. Si une erreur survient dans la table *Gold*, un ingénieur peut remonter visuellement le graphe jusqu'à la source (Postgres) pour identifier instantanément le script coupable. Cela réduit le temps de débogage (MTTR) de plusieurs jours à quelques minutes.

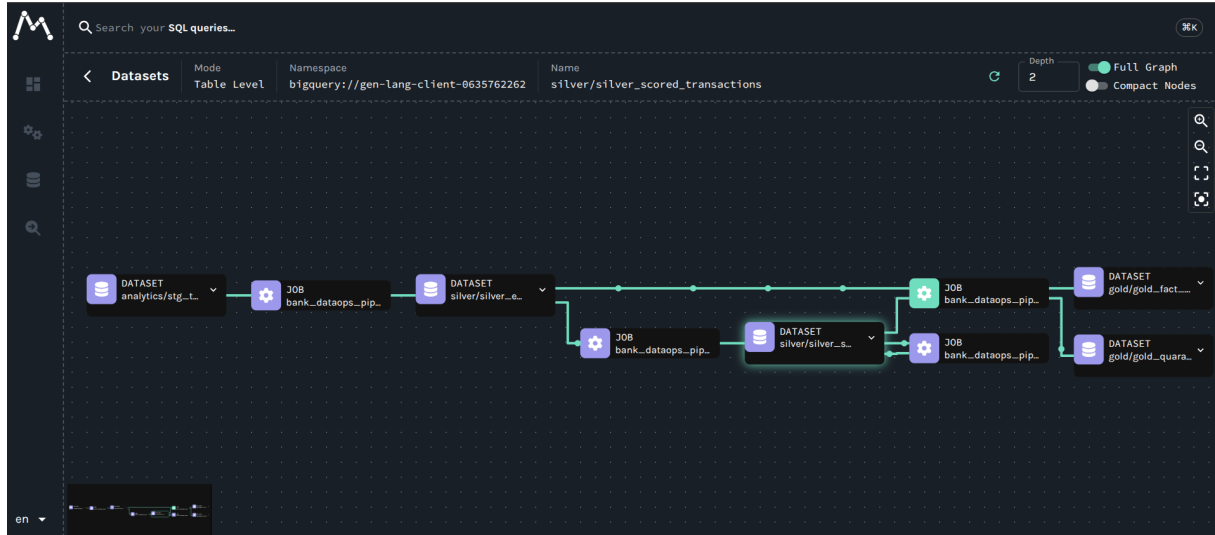


FIGURE 7 – Graphe de traçabilité (Data Lineage) généré par Marquez

5 Restitution Visuelle : Tableaux de Bord

Les données de haute qualité de la couche Gold sont enfin exploitées par **Apache Superset**. Nous avons conçu deux tableaux de bord distincts pour deux personas métiers différents.

5.1 Le Tableau de Bord de Performance Financière

Destiné à la direction et aux analystes financiers, ce tableau de bord se branche exclusivement sur la table des faits saine. Il présente les indicateurs clés de performance (KPIs) de la banque (volumes de transactions réelles, répartition des canaux de vente, etc.). Les utilisateurs ont la certitude mathématique que les données présentées ne contiennent ni erreurs de base de données, ni fraudes.

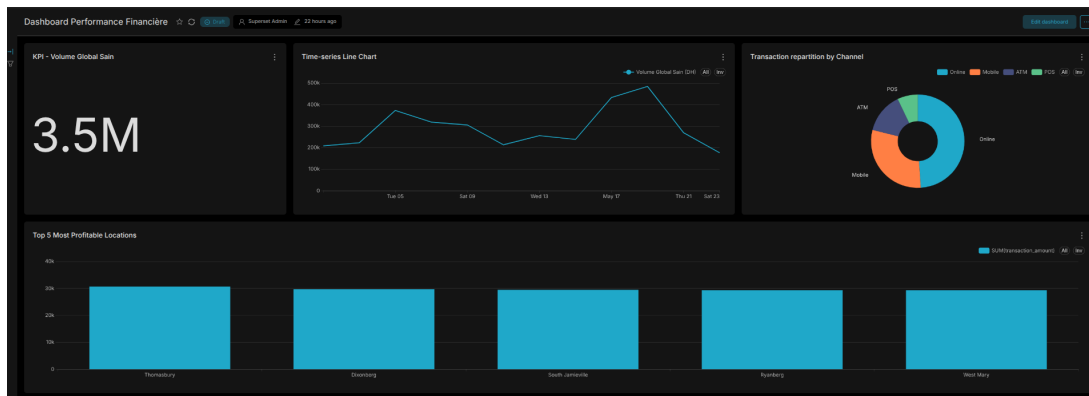


FIGURE 8 – Tableau de bord de Performance Financière

5.2 Le Tableau de Bord des Alertes Fraudes

Destiné aux équipes de sécurité et d'investigation, ce tableau de bord est branché sur la table de quarantaine. Il permet de visualiser en temps réel le nombre de fraudes détectées par les algorithmes de Machine Learning, leur nature et leur géographie.

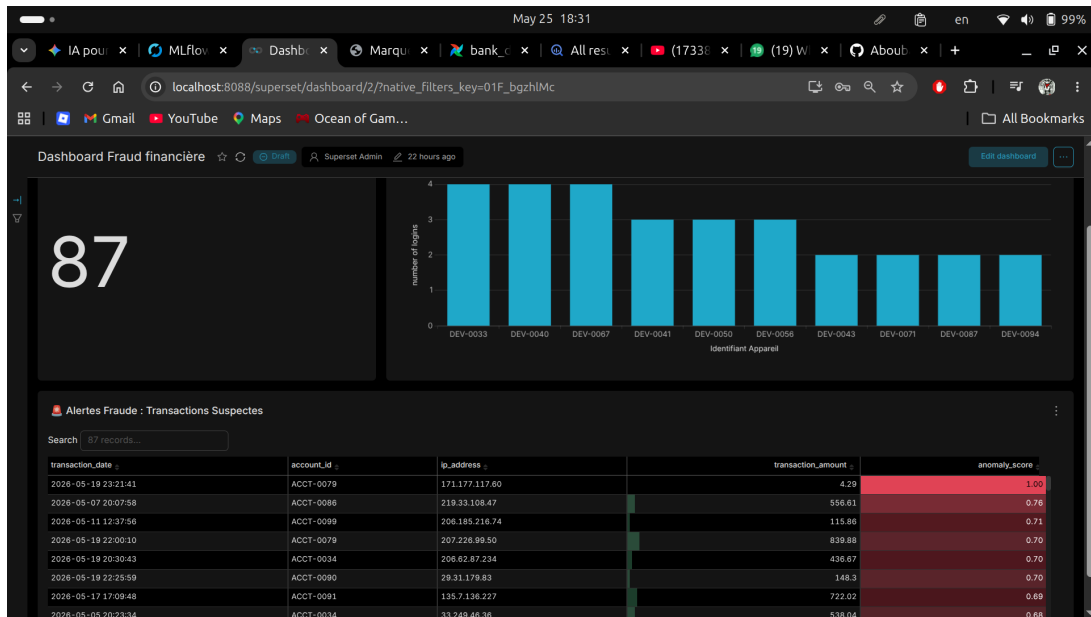


FIGURE 9 – Tableau de bord de Monitoring de la Fraude

6 Conclusion

Le Projet 13 initial nous demandait d'établir un "tableau de bord de qualité de données avec profilage et traçabilité". En réponse à cette problématique critique pour les DSI modernes, nous n'avons pas seulement répondu au cahier des charges, nous avons conçu et déployé une **Plateforme DataOps d'Entreprise**.

Nous avons réussi à intégrer un écosystème complexe d'outils modernes (Airflow, Great Expectations, dbt, BigQuery, Marquez, Superset) de manière fluide et robuste.

Les résultats obtenus sont probants :

1. **Fiabilité totale** : Les pannes silencieuses (erreurs structurelles) sont arrêtées par Great Expectations dès l'extraction.
2. **Intelligence Artificielle intégrée** : L'utilisation de réseaux de neurones (Autoencoders) et d'arbres d'isolation, pilotés par MLflow, nous permet de repérer les fraudes subtiles que l'humain et les règles de gestion laissent passer.
3. **Transparence et Débogage rapide** : L'implémentation de Marquez fournit un graphe de lignage permettant une investigation instantanée des problèmes.
4. **Gouvernance des données** : La séparation claire en architecture Medaillon (Bronze, Silver, Gold) garantit que les consommateurs finaux de la BI utilisent toujours une donnée certifiée.

Ce projet démontre qu'une stratégie d'observabilité des données ne consiste pas simplement à ajouter des tests à la fin d'un pipeline, mais nécessite une approche d'ingénierie préventive, intégrant la qualité, la traçabilité et le monitoring MLOps dès la conception de la plateforme.